Software Requirements Specification (SRS)

EduAirControl

Version: 1.2 — documentary review and alignment with the submitted repository **Date:** October 9, 2026 **Project:** EduAirControl **Prepared for:** Servicio Nacional de Aprendizaje (SENA), Software Analysis and Development — cohort 3145556 **Team members:** María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas **Instructor:** José de Jesús Motta Vargas

> ***Status convention.*** *"Implemented," "Partial," and "Pending" reproduce the status stated in the source specification and repository documentation. They do not, by themselves, constitute independent certification that a feature works. Quantitative quality targets are proposed acceptance criteria and must be verified with test evidence.*

Document control

Version	Date	Description
1.0	2025	Initial SRS version.
1.1	2026-10-08	Updated against web, mobile, and microservices documentation.
1.2	2026-10-09	Structure, terminology, traceability, statuses, and documentary discrepancies reviewed; implemented capabilities are distinguished from targets still requiring verification.

1. Introduction

1.1 Purpose

This document specifies EduAirControl's functional and non-functional requirements, interfaces, constraints, and acceptance criteria. It is a shared reference for development, testing, deployment, maintenance, and product evaluation.

1.2 Problem statement

In educational environments, temperature, humidity, carbon dioxide (CO■) concentration, and noise can affect comfort and learning conditions. When these variables are not continuously observed, institutional staff have limited information to detect abnormal conditions and decide what actions to take.

1.3 Product purpose

EduAirControl is an IoT-based school environmental monitoring platform. It receives sensor measurements, stores them, presents current and historical values, and generates alerts based on configured thresholds. It supports informed decisions about educational spaces; it does not replace certified instruments and is not an automatic environmental control system.

1.4 General objective

Develop a school environmental monitoring system that records and displays temperature, CO■, humidity, and noise to support student and teacher well-being and decision-making in educational institutions.

1.5 Specific objectives

- Receive and query environmental measurements associated with educational spaces.
- Present current and historical information through web and mobile interfaces.
- Report normal, warning, and critical conditions.
- Allow institutions, campuses, environments, users, sensors, and preferences to be managed according to permissions.
- Provide accessibility and visual customization options.
- Enforce institutional data isolation as a security requirement.

2. Scope

2.1 Included in the documented MVP

- Authentication and role-based access control.

- Email/password registration and login, and social authentication where configured.
- Management of institutions, campuses, users, and roles.
- Educational environment management.
- Sensor registration/administration and sensor-to-environment associations, with some capabilities still partial.
- Environmental measurement ingestion and storage.
- Current and historical measurement queries.
- Dashboards, filters, favorites, profile, and preferences.
- Alerts based on temperature, humidity, CO■, and noise thresholds.
- Mobile Bluetooth device provisioning, subject to device compatibility.
- Web and mobile applications.

2.2 Out of scope

- Advanced environmental prediction or predictive models.
- Automatic control of ventilation, air conditioning, or other equipment.
- Integration with external environmental automation platforms.
- Design and manufacture of sensor hardware.
- Metrological accuracy guarantees for non-certified sensors.
- External push/email notifications as a confirmed capability; these remain pending.

2.3 Product perspective

The repository documents a distributed architecture with React/Vite web and Expo/React Native mobile clients, Java/Spring Boot services, an API Gateway, PostgreSQL, Redis, and RabbitMQ, as well as Docker containers and integration tooling. Security, user management, classroom/environment management, sensor management, environmental monitoring, and user-experience services are identified. The actual availability of each capability must be confirmed in the deployed version.

Alignment note: the source SRS mentions HTTPS or MQTT for telemetry, whereas the architecture documentation describes HTTP ingestion and the monitoring service contains an HTTP controller. The reviewed repository did not provide sufficient evidence to declare MQTT an operational MVP channel. Therefore, HTTP/HTTPS is the documented baseline; MQTT must be confirmed before it is considered implemented.

3. Users and actors

Actor	Expected responsibilities / permissions
User (`USER`)	Views authorized environments and variables, history, alerts, favorites, profile, and preferences.
Administrator (`ADMIN`)	Manages environments, sensors, and thresholds within the authorized institutional scope.
Super administrator (`SUPER_ADMIN`)	Manages institutions and global user/role administration.
IoT device	Sends identified measurements for validation and storage.
External identity provider	Google/Facebook, if credentials and callbacks are configured.
Technical operator	Deploys, configures, monitors, and recovers services; not an end-user functional role.

Institution and campus isolation is mandatory. The documented rule allows an administrator to view the authorized institution while other users are restricted to their campus. Full implementation of this isolation is marked partial and requires authorization tests.

4. Assumptions, dependencies, and constraints

4.1 Assumptions

- Educational spaces have sufficient power and Wi-Fi connectivity for devices.
- Sensors are installed and configured correctly.

- Institutions assign staff responsible for reviewing alerts.
- Users receive basic guidance on using the platform.
- Sensor measurements are sufficiently accurate for the intended advisory use.

4.2 Dependencies

- Email service for password recovery.
- Google and Facebook for social authentication, if enabled.
- PostgreSQL persistence; Redis for documented auxiliary functions; RabbitMQ for inter-service events where applicable.
- Docker and the runtime infrastructure.
- GitHub/Jenkins for source control and automation when connected to the deployment environment.
- ESP32 devices and sensors, plus Bluetooth/BLE for mobile provisioning.

4.3 Constraints

- Connectivity is required to query recent data and communicate with services.
- Data must be segregated by institution and campus according to permissions.
- Secrets and passwords must not be stored in source code or written to logs.
- The web app must work in modern browsers; the mobile app depends on versions supported by the project's Expo configuration.
- Threshold values must be centralized and consistent across backend, web, and mobile.
- This specification does not claim that all performance, availability, coverage, or recovery targets have already been measured.

5. External interfaces

5.1 User interface

- Web and mobile interfaces with cards, lists, forms, environmental indicators, and historical views.
- Light and dark modes.
- Display options for protanopia, deuteranopia, and tritanopia, and adjustable font sizes, as described in the source SRS.
- Planned languages: Spanish, English, French, and Portuguese.
- Documented base colors: light `#f0fafa` and dark `#0a0f1e`; these refer to base backgrounds, not background images.
- Full feature parity between web and mobile is not assumed and must be verified per screen and workflow.

5.2 Hardware

- Internet-connected computers and mobile devices.
- ESP32-based measurement devices and temperature, humidity, CO■, and noise sensors.
- Bluetooth Low Energy (BLE) on mobile for provisioning compatible devices.

5.3 Software and communication

- Web client: React/Vite, according to the repository.
- Mobile client: Expo/React Native; the exact SDK version must be confirmed in project manifests.
- Backend: Java/Spring Boot microservices.
- Persistence: PostgreSQL; Redis and RabbitMQ according to configuration.
- HTTP/REST APIs behind an API Gateway. HTTP/HTTPS sensor ingestion is the documented baseline. MQTT is not confirmed until executable configuration and evidence are available.
- Social authentication through OAuth2/OIDC only if providers, credentials, and redirects are configured.

6. Functional requirements

Priority convention: High = critical to the main workflow or security; Medium = important for the MVP; Pending = must not be advertised as available. Statuses below come from the source specification and reviewed documentation.

FR1. Authentication and access

ID	Requirement	Status
FR1.1	Allow email/password login and issue access and refresh tokens according to configuration. The source specifies a 1-hour access token and a 7-day refresh token.	Implemented in source
FR1.2	Provide a second authentication factor.	Pending
FR1.3	Allow registration with username, email, password, institution code, and optional campus; reject duplicate email addresses.	Implemented in source
FR1.4	Allow Google/Facebook authentication through OAuth2 when configured; link existing accounts or start onboarding for new accounts.	Implemented in source; configuration must be verified
FR1.5	Require acceptance of terms and conditions during registration.	Implemented in source
FR1.6	Allow password recovery through an emailed code, validation, and password reset; allow code resending.	Implemented in source
FR1.7	Allow authenticated users to change their password and request account deletion with confirmation.	Implemented in source
FR1.8	Restrict protected routes and operations by `USER`, `ADMIN`, and `SUPER_ADMIN`; return 401 when unauthenticated and 403 when unauthorized.	Implemented in source
FR1.9	Allow logout and token renewal; revocation must invalidate a token according to the configured strategy.	Implemented in source

FR2. Institutions, campuses, and users

ID	Requirement	Status
FR2.1	Allow super administrators to create, view, and edit institutions with unique codes.	Implemented in source
FR2.2	Allow campuses associated with an institution to be created, viewed, edited, and deleted.	Partial
FR2.3	Isolate data by institution and campus across tokens, gateway, and service queries. Users must not access another institution's data.	Partial; security tests required
FR2.4	Allow super administrators to view users and assign roles through administration functions.	Implemented in source

FR3. Dashboard, variables, and analysis

ID	Requirement	Status
FR3.1	Show authorized environments and their environmental status, with access to details.	Implemented in source
FR3.2	Show temperature, humidity, CO■, and noise with value, unit, and status.	Implemented in source
FR3.3	Calculate a 0–100 score and rank environments according to threshold compliance.	Partial / outside the initial MVP; scope must be confirmed
FR3.4	Query current and historical data by day, week, month, and year, selecting a period and variable.	Partial
FR3.5	Display charts or indicators to compare measurement trends.	Analysis objective; verify per client

FR4. Search and filtering

ID	Requirement	Status
FR4.1	Search environments by name and allow viewing, recording, or clearing search history if the service is available.	Partial
FR4.2	Filter by building/block, floor, favorites, and environmental status. Variable-based search remains pending in the source.	Partial

FR5. Environment details and ratings

ID	Requirement	Status
FR5.1	Show location, capacity, type, environmental quality, current metrics, reference ranges, variable explanations, and basic recommendations.	Implemented in source
FR5.2	Allow users to rate an environment from 1 to 5 and enter a comment up to 1,000 characters.	Partial: source says the screen and moderation are missing
FR5.3	Allow users to view a variable's detail, current value, status, and history.	Partial

FR6. Alerts and notifications

ID	Requirement	Status
FR6.1	Compare each measurement with active thresholds and classify deviations as `WARNING` or `CRITICAL`.	Implemented in source
FR6.2	List alerts and allow users to acknowledge them.	Implemented in source
FR6.3	Show notifications chronologically, open related items, and mark them read per user. The source warns that part of the UI calculates notifications from fixed client-side values and should consume backend alerts.	Partial
FR6.4	Allow users to enable/disable alerts, warnings, daily summaries, and sound, according to available preferences.	Implemented in source
FR6.5	Send alerts outside the application through push notification or email to responsible staff.	Pending

FR7. Profile, favorites, and preferences

ID	Requirement	Status
FR7.1	Allow editing of name, job title, phone, location, and profile photo.	Implemented in source
FR7.2	Allow users to favorite/unfavorite environments and view their own list. Variable favorites are not included.	Implemented in source
FR7.3	Provide dark mode, three color-vision options, and three font sizes.	Implemented in source
FR7.4	Provide Spanish, English, French, and Portuguese.	Implemented in source
FR7.5	Save date format, time zone, and reminder preferences; support local storage while offline where implemented.	Implemented in source

FR8. Environment, sensor, and threshold administration

ID	Requirement	Status
FR8.1	Allow administrators to create, edit, and delete environments with name, location, floor, capacity, and type.	Implemented in source
FR8.2	Register, edit, enable/disable, and remove sensors; associate measured variables; install/remove sensors from environments; validate required fields.	Partial; source says the web sensor panel uses sample data
FR8.3	Configure normal, warning, and critical thresholds by variable and environment type, with effective dates; close validity periods instead of deleting history.	Partial

FR9. IoT and measurements

ID	Requirement	Status
FR9.1	Receive, validate, and persist measurements; prevent duplicates on retries. The documented baseline is HTTP/HTTPS.	Implemented in source
FR9.2	Retain measurements for historical queries by day, week, month, and year.	Implemented in source
FR9.3	Query the latest value for each variable in an environment.	Implemented in source
FR9.4	Allow administrators to configure a compatible ESP32 device from the mobile app over BLE, requesting permissions and sending network configuration.	Implemented in source; validate hardware and client
FR9.5	Register devices with MAC address, name, type, firmware, network, and status, and assign them to environments.	Partial; source did not confirm a backend service for `/api/v1/devices`

Source status summary

Status	Reported count	Note
Implemented	26	Original count; not an independent audit
Partial	12	Gaps must be closed and evidence supplied
Pending	2	Second factor and external notifications
Reported total	**40**	The source table summarizes sub-requirements; this version reorganizes and clarifies some compound requirements. Do not use this count as an acceptance metric without normalizing the traceability matrix.

7. Environmental variables and thresholds

The following ranges are documented in the source SRS for each environment type. They should be centralized in one configuration and validated with the institution's responsible staff and actual sensors. They are not medical advice or universal regulatory standards.

7.1 Valid input ranges

Variable	Code	Unit	Valid range
Temperature	`TEMP`	°C	-10 to 50
Relative humidity	`HUM`	%	0 to 100
Carbon dioxide	`CO2`	ppm	0 to 5,000
Noise level	`NOISE`	dB	0 to 140

7.2 Classroom (CLASSROOM)

Variable	Normal	Warning	Critical
Temperature	20–26 °C	18–20 °C or 26–28 °C	<18 °C or >28 °C
Humidity	40–60%	30–40% or 60–70%	<30% or >70%
CO■	0–800 ppm	800–1,000 ppm	>1,000 ppm
Noise	0–45 dB	45–55 dB	>55 dB

7.3 Laboratory (LABORATORY)

Variable	Normal	Warning	Critical
Temperature	18–24 °C	16–18 °C or 24–26 °C	<16 °C or >26 °C
Humidity	30–60%	25–30% or 60–75%	<25% or >75%
CO■	0–600 ppm	600–800 ppm	>800 ppm
Noise	0–50 dB	50–60 dB	>60 dB

7.4 Library (LIBRARY)

Variable	Normal	Warning	Critical
Temperature	20–24 °C	18–20 °C or 24–26 °C	<18 °C or >26 °C
Humidity	40–55%	35–40% or 55–65%	<35% or >65%
CO■	0–600 ppm	600–800 ppm	>800 ppm
Noise	0–40 dB	40–50 dB	>50 dB

Interpretation rule: shared boundary values (for example, 800 ppm) require one precedence rule so a single value cannot be classified into two states. The source does not explicitly define inclusive/exclusive endpoints for every interval; this must be fixed in ADR-014/code and covered by boundary tests.

8. Non-functional requirements

The following are measurable targets documented in the quality strategy. Unless test evidence exists, they must be treated as targets awaiting verification, not as achieved results.

ID	Category	Objective / acceptance criterion
NFR1	Performance	P95 ≤500 ms for critical endpoints under expected load; historical analysis P95 ≤2 s; initial dashboard data P95 ≤3 s; ingestion acknowledgement P95 ≤1 s; error rate <1% excluding client validation errors.
NFR2	Availability and resilience	Monthly availability ≥99.5% during the agreed service window; liveness/readiness health endpoints; controlled errors with correlation IDs; idempotent retries.
NFR3	Scalability	Stateless horizontal scaling; initial target up to five instances per service; increase capacity within two minutes during a sudden growth scenario, subject to infrastructure.
NFR4	Security and privacy	Authentication on private routes; role and tenant authorization; input validation; TLS in deployed environments; secrets outside source code; no passwords or full tokens in logs.
NFR5	Observability	Structured logs with timestamp, service, level, and correlation ID; metrics and traces; operational alerts; telemetry outages must not stop ingestion.

ID	Category	Objective / acceptance criterion
NFR6	Maintainability and documentation	Target ≥80% coverage of measurable code and ≥90% of business logic; cyclomatic complexity ≤10 per function where reasonable; reproducible setup in under one hour; current OpenAPI contract for each exposed API.
NFR7	Portability and deployment	Services deployable with Docker; environment-variable configuration; no fixed local paths or embedded secrets; Compose is the minimum documented target.
NFR8	Recovery	Target RTO <5 min for a microservice/gateway failure and <30 min for a database failure; target RPO ≤24 h with daily backups. Targets require operational confirmation and restore testing.
NFR9	Usability, accessibility, and languages	Understandable main workflows; Spanish, English, French, and Portuguese; light/dark modes and color-vision options; keyboard navigation and WCAG 2.1 AA target for key workflows; show data age when offline.
NFR10	Compatibility and responsiveness	Modern Chrome, Firefox, Edge, and Safari; Android/iOS versions supported by the project's Expo SDK; responsive layout from 360×640 px to 1280×720 px without loss of core functions or horizontal overflow.

8.1 Reference critical endpoints

The quality strategy lists, among others, `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/users/{id}`, `GET /api/environments/{id}`, `GET /api/monitoring/current`, and `GET /api/monitoring/history` as critical. Actual routes must be verified against the deployed controllers and API Gateway; these examples are not a final OpenAPI contract.

9. Use cases

ID	Use case	Primary actor	Related requirements
UC-01	Register	User	FR1.3, FR1.5
UC-02	Log in with email/password	User, Administrator, Super administrator	FR1.1, FR1.8
UC-03	Log in with Google or Facebook	User	FR1.4
UC-04	Recover or change password	User	FR1.6, FR1.7
UC-05	View dashboard and environment ranking	User	FR3.1–FR3.3
UC-06	Search and filter environments	User	FR4.1, FR4.2
UC-07	View environment details and variables	User	FR5.1, FR5.3
UC-08	View historical analysis	User	FR3.4, FR9.2
UC-09	Rate an environment	User	FR5.2
UC-10	Manage favorites	User	FR7.2
UC-11	Review alerts and notifications	User	FR6.1–FR6.4
UC-12	Manage profile and preferences	User	FR7.1, FR7.3–FR7.5
UC-13	Manage environments	Administrator	FR8.1
UC-14	Manage sensors and thresholds	Administrator	FR8.2, FR8.3
UC-15	Configure a device over Bluetooth	Administrator	FR9.4, FR9.5
UC-16	Manage institutions, campuses, and users	Super administrator	FR2.1–FR2.4
UC-17	Send measurements	IoT device	FR9.1

9.1 General ingestion flow

1. The device reads a value and associates it with a sensor, variable, and timestamp.
2. It sends the measurement through the configured channel; HTTP/HTTPS is the documented baseline.
3. The service validates structure, range, device identity, and duplicates according to enabled capabilities.
4. The accepted measurement is stored and made available for current/historical queries.
5. Monitoring logic compares the value with active thresholds and creates or updates an alert when appropriate.
6. Clients query the value and display its status, unit, and update time.

10. Business rules and validation

- Every protected operation requires authentication and role-based authorization.
- Queries must be restricted to the authorized institution and campus.
- Measurements must include enough identification to associate them with a sensor, variable, and environment.
- Values outside a variable's valid range must be rejected or explicitly marked invalid according to a defined policy; they must not be silently mixed with valid measurements.
- Retried measurements must not create duplicates.
- Thresholds must have effective periods and preserve change history.
- `WARNING` means deviation from the normal range within the warning band; `CRITICAL` means reaching the critical limits defined for the environment type.
- Authentication and authorization failures must be distinguishable (401 and 403).
- Credentials, secrets, and full tokens must not appear in logs or error messages.
- Measurement timestamps and time zones must be handled consistently for historical analysis.

11. Global acceptance criteria

A release may be accepted only when verifiable evidence is available for:

1. Registration, authentication, logout, and role-based authorization.
2. Institution/campus isolation, including unauthorized access attempts.
3. Environment administration and environmental value queries.
4. Measurement ingestion, validation, persistence, and duplicate prevention on retries.
5. Alert classification tests for normal, warning, critical, and boundary values.
6. Historical queries and correct date-range behavior.
7. Web/mobile interfaces and declared preferences, with functional differences documented.
8. Reproducible deployment, externalized configuration, and backup recovery.
9. Performance, security, accessibility, and compatibility evidence for applicable NFR targets.
10. Traceability from requirements to test cases, results, defects, and fixes.

12. Traceability and change management

- Functional requirements use `FRx.y`; non-functional requirements use `NFRx`; use cases use `UC-xx`.
- The traceability matrix must link each requirement to its user story, responsible service, test case, evidence, and status.
- Scope, threshold, role, API, or authentication changes must update this document, the traceability matrix, API contracts, and affected manuals.
- Threshold changes must be reviewed for their impact on backend, web, mobile, tests, and historical data.

13. Risks and open points

1. Complete and test institution/campus isolation (FR2.3).
2. Resolve the discrepancy between MQTT in the earlier SRS and HTTP ingestion in the architecture/code documentation.

3. Connect notifications to real backend alerts and define external notifications (FR6.3/FR6.5).
4. Complete actual sensor/device administration (FR8.2/FR9.5) and remove sample data from operational paths.
5. Formally define inclusive/exclusive threshold boundaries and test edge values.
6. Provide reports for NFR targets: load, coverage, recovery, security, accessibility, and compatibility.
7. Confirm API paths against the controllers and gateway in the release to be deployed.
8. Verify Google/Facebook and email credentials/configuration before advertising those flows as available.

14. Glossary

- **API Gateway:** entry point routing requests to services.
- **BLE:** Bluetooth Low Energy.
- **CO■:** carbon dioxide.
- **IoT:** Internet of Things.
- **MVP:** minimum viable product.
- **OAuth2/OIDC:** authorization and federated identity protocols/standards.
- **P95:** 95th percentile of response time.
- **RPO:** recovery point objective; target maximum data loss.
- **RTO:** recovery time objective.
- **SRS:** Software Requirements Specification.
- **Tenant:** logical institution scope in a multi-institution platform.
- **Threshold:** limit used to classify a measurement.

15. Documentary references

- EduAirControl SRS, source version 1.1, supplied as `SRS_EduAirControl(1).docx`.
- Submitted repository `proyecto.zip`, especially `Docs-ea-control/04-requirements/`, `05-architecture/overview.md`, `08-uml/diagram-index.md`, `09-microservices/service-catalog.md`, `11-quality/testing-strategy.md`, and `microservicios/ms-environment-monitoring/`.
- IEEE 830, as a historical reference for the document structure; future revisions should maintain verifiable and traceable requirements.
- World Wide Web Consortium (W3C), Web Content Accessibility Guidelines (WCAG) 2.1, referenced as an accessibility target in the source.
- Environmental references cited in the original SRS should be verified for title, year, availability, and exact relationship to the thresholds before being used as normative support.